IC Lab Formal Verification
Bonus Report 2024 Fall

Name: 邱琦閎 **Student ID:** 313510227 **Account:** iclab047

(a) What is Formal verification?
What's the difference between **Formal** and **Pattern** based verification?
And list the pros and cons for each.

1. **What is Formal verification?**
   Formal verification 與 Pattern based verification 之間的主要差異在於是否需要 test stimuli。在 Formal verification 中，無需撰寫測試刺激；相反，Pattern based verification 需要在測試模式中撰寫 test stimuli，並根據輸入信號檢查輸出信號。由於人為錯誤，這種方法可能會遺漏某些極端情況。然而，形式驗證無需測試刺激即可檢查設計中的所有可能狀態和轉換，並識別與規範或屬性相關的錯誤。這有助於發現可能被忽略的極端情況，並節省驗證過程中的大量精力。
2. **Formal verification**：Formal verification 是使用數學方法驗證所有可能的輸入組合，驗證的過程中並不會在每個 cycle 進行定時檢查，而是根據設定的 specifications 驗證所有未驅動線和輸入值的所有組合。此外，它會在第一個 cycle 測試所有未初始化寄存器的值的所有組合。然後，將所有違反 assertion properties 的情況和 cover properties 紀錄後生成報告，從而允許在設計流程中及早發現問題並更有效地進行調試和修復。

2. **pros and cons for each.**

	ADVANTAGE	DISADVANTAGE
Formal verification	Formal verification 的驗證結果更加可靠，因為驗證過程中嘗試過所有可能的狀態，所以可以確保設計功能在不同情況下都是正確的。此外，由於沒有獲只有一點的隨機，帶來導致系統化和確定性的特徵。設計者可以在設計 testbench 和邏輯模擬之前開始編寫 SVA property，接著使用 EDA tool 生成 input 來完全測試設計。	需要全面探索所有可能性，這可能導致驗證所需檢查數量的指數增長。隨著設計複雜度的增加，驗證所需的時間也會成比例地增長，這可能會導致資源消耗顯著增加。此外，當環境約束 underconstrained/overconstrained，可能會產生誤報或漏報錯誤，驗證品質對 testbench 質量的依賴性。
Pattern verification	Pattern verification 消耗較少的資源，使其適用於更	由於可能發生覆蓋不完全，所以 Pattern verification 的準確性不如

	廣泛的應用，包括同時進行的內部信號組合（計算資源較少）。設計者可以控制要驗證的模式數量，且驗證時間或週期比 Formal verification	Formal verification。自動化程度較低且時間消耗增加。此外，若希望驗證 corner case 則 Pattern verification 相較於 Formal verification 會有更高的複雜性和揹難度(有時我們使用隨機化來生成刺激)。

(b) Explain SVA (SystemVerilog Assertions) and the roles of Assertion, Cover, and Assumption. What is glue logic?
Why will we use **glue logic** to simplify our SVA expression?

Explain SVA (SystemVerilog Assertions) and the roles of Assertion, Cover, and Assumption.

SystemVerilog Assertions (SVA) 是一種被設計來描述和解釋 properties 的語言，主要用於描述和檢查設計的行為。SVA 提供了三種主要的驗證構造：Assertion、Cover 和 Assumption。

1. Input restrictions with “assume”
2. Checker property with “assert”
3. Stimulus coverage with “cover”

Assertion (斷言)：

- Check something that should never happen IF a condition is met.
  For example: Check if a signal is at a high level at a specific time.
  assert property (@(posedge clk) signal == 1);

Cover (覆蓋)：

- Cover is used to collect statistical data during the design's operation, ensuring that all possible scenarios are tested. Cover points can help designers understand the completeness of the testing.
- For example, check if a signal reaches all possible states at different times.
  cover property (@(posedge clk) signal == 1)
- In both formal verification and dynamic simulation, cover is used to check the completeness of the verification work. Similar to assertions, cover is also a type of verification target. However, unlike assertions, cover does not require the condition to always be true in all verification scenarios. As long as the cover condition can be true in some verification scenarios (even if it only appears once), the cover verification result is considered passed; otherwise, the verification fails.

Assumption (假設)：

- Not all inputs are reasonable. Assume properties are used to restrict the input combinations considered in formal verification (assumptions are true). During the verification process, input combinations where assumptions are false will not be considered.

What is glue logic?

Glue logic is an auxiliary logic to help to observe and track events. It greatly simplify coding.

Why will we use glue logic to simplify our SVA expression?

Modeling complex behaviors, SVA may be complicated, so designer can ues auxiliary logic to observe and track events (glue logic). It greatly simplify coding. Once glue logic is in place, expressing SVA properties may be trivial.

Glue logic comes at no extra price

- JasperGold does not care whether property is all SVA or SVA+glue logic
- Recommendation is to choose based on clarity

(c) What is the difference between **Functional coverage** and **Code coverage**?
What's the meaning of 100% code coverage, could we claim that our assertion is well enough for verification? Why?

Functional Coverage

Functional Coverage usually planned by designers, involves validating design functionalities through assertions and coverage groups, which requires manual planning when designing coverpoints. It also requires designers to engage in coding and debugging, making it a time-consuming process. It is possible to represent all meaningful design functionality. Moreover, due to its manual nature, there's a significant risk of incomplete verification resulting from human own errors, leading to incomplete coverage and longer design times(overlook certain corner cases…).

Code Coverage

Code coverage is a metric used to measure the extent to which the source code of a program is executed during testing by EDA tool. It aims to ensure that every line of code is triggered(Can't guarantee complete functional correctness.). It is easy to generate automatically and is guaranteed to be structurally complete, reducing the likelihood of human error. Furthermore, it may overlook issues such as unmeaningful coding(some cases may inherently not be reachable, requiring manual waiver of these alarms.).

What's the meaning of 100% code coverage, could we claim that our assertion is well enough for verification? Why?

No, certainly not. This does not ensure the correctness of your RTL code. To verify the functional correctness of the design, Functional or Checker Coverage must be used for validation. Code coverage merely verifies that each line of code in the design has been executed and does not guarantee the functional correctness of the design. Thus, achieving 100% code coverage only means that every line of code in the design has been executed during testing. Even with 100% code coverage, there is still a chance that the functionality is not correct.

(d) What is the difference between **COI coverage** and **proof coverage** for realizing checker's Functional Coverage completeness? Try to explain from the meaning, relationship, and tool effort perspective.

	COI coverage	proof coverage
meaning	The region refers that affect this assertion when a designer is writing assertions and affects the cover items checked in the formal environment. After finding the union of the all assertion COIs, the remaining Out of COI cover items indicate holes in the assertion set – code that is not checked by any asserts.	the region cannot truly influence assertion status. It represents the portion of the design verified by formal engines.
relationship	Proof coverage is a subset of the COI. COI represents the maximum potential of proof coverage COI doesn't require a proof to take place, while proof coverage does	
tool effort perspective	Fast measurement – no formal engines are run. It's faster than proof coverage because COI is the area formed by all cover items that affect the output, which just needs to trace forward continuously, without the need for a formal engine.	It identify more unchecked code than COI measurement, with greater tool effort(Slower measurement than COI). Running Proof Coverage involves conducting formal proof.

(e) What are the roles of **ABVIP** and **scoreboard** separately?
Try to explain the definition, objective, and the benefit.

	ABVIP	**scoreboard**
definition	The Assertion Based Verification Intellectual Properties(ABVIPs) are a comprehensive set of checkers and RTL that check for protocol compliance. (e.g. ARM AMBA AXI based systems and designs)	Scoreboard behaves like a monitor – Observe the input data, progress, correctness, and output data of the DUV of the system during simulation.

objective	Provides a well-developed tool for designers to perform detailed checks on existing protocols. ABVIP aims to enhance the efficiency of meeting expected functionality and behavior	Verify the consistency between inputs and outputs at a high level of design. This process involves comparing expected results with actual simulation outcomes to identify errors in the design. It can detect issues such as dropped, duplicated, corrupted data packets, and ensure the correct order of data packets.
benefit	Reduce the time spent on designing assertions and ensure their accuracy. Utilizing ABVIP can greatly decrease the time needed to create checkers and minimize the probability of errors.	Ensure that the design performs correctly under various conditions, which streamlines the verification process, enhances testing efficiency, and minimizes the risk of human errors.

(f) Among the JasperGold tools (Formal Verification, SuperLint, Jasper CDC, IMC Coverage), which one do you think is the most effective based on its functionality and typical application scenarios? Please explain your reasoning by describing a hypothetical scenario where this tool would be particularly beneficial, and discuss any potential challenges or limitations that might arise when using it.

Formal verification is the most effective tool that I believe it is.

Hypothetical Scenario
Imagine we are designing a high-reliability, safety-critical vehicle control system. In this scenario, every part of the design must strictly adhere to specifications and be free of errors. Traditional simulation methods may not cover all possible input combinations and state transitions, potentially leaving design flaws undetected.

Why Formal Verification is particularly beneficial
Formal verification uses mathematical methods to verify the correctness of the design, capable of checking all possible states and input combinations to ensure the design meets specifications under all conditions. This is especially important for high-reliability and safety-critical systems, as it can identify and correct potential errors early in the design process, avoiding costly fixes later on.

Potential Challenges and Limitations

- **Computational Resources:**
  Formal Verification requires significant computational resources to handle complex designs and state spaces. For very large designs, the verification process can be extremely

time-consuming or even infeasible.

- **Design Abstraction:**
  To make Formal Verification feasible, designers may need to abstract and simplify the design. This can lead to certain details being overlooked, potentially affecting the accuracy of the verification results.
- **Learning Curve:**
  Using Formal Verification tools requires specialized knowledge and experience. Design teams may need to invest time and resources to learn how to use these tools effectively.